

Atividade — Backend (API) do E-Cycle

Construindo a API em Node.js + Express, pasta por pasta

Escola: E.E. José Domingos da Silveira · **Turma:** 3º A — DS
Projeto: E-Cycle (Projeto Multidisciplinar) · **Matéria:** Backend

Aluno(a): _____ Data: ____ / ____ / ____

Objetivo

Construir o **backend** (a API) do E-Cycle. Você vai criar cada pasta e cada arquivo, digitando (ou colando) o código que está neste PDF. Ao final, a API roda no seu computador e conversa com o banco que você criou no Supabase.

Pré-requisitos (fases anteriores)

- **Versionamento:**
você já tem o repositório do projeto (pasta `e-cycle`).
- **Banco de Dados:**
você já criou as tabelas no Supabase e tem a **string de conexão** e a **senha** do banco.
- **Node.js**
instalado — confira com `node -v`.

Ordem importa

Os comandos `npm install` e `npm start` só funcionam **depois** que os arquivos existirem. Por isso eles ficam no **fim** desta atividade. Não rode `npm` em pasta vazia.

Parte 1 — Criar a estrutura de pastas e o package.json

Passo 1. No terminal, entre na pasta do projeto e crie as pastas do backend:

- **Windows (PowerShell):**

```
cd e-cycle
mkdir backend\src\config, backend\src\middlewares, backend\src\services, backend\src\controllers, backend\src\routes
```

- **Linux / Git Bash:**

```
cd e-cycle
mkdir -p backend/src/config backend/src/middlewares backend/src/services backend/src/controllers backend/src/routes
```

Passo 2. Abra a pasta `e-cycle` no VSCode. Crie o arquivo `backend/package.json` e cole:

backend/package.json

```
{
  "name": "e-cycle-backend",
  "version": "1.0.0",
  "description": "API do E-Cycle - plataforma de reciclagem de eletronicos",
  "main": "src/server.js",
  "type": "commonjs",
  "scripts": {
    "start": "node src/server.js",
    "dev": "node --watch src/server.js"
  },
  "dependencies": {
    "bcrypt": "^5.1.1",
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "jsonwebtoken": "^9.0.2",
    "pg": "^8.12.0"
  }
}
```

Passo 3. Crie também o arquivo `backend/.env.example` (o modelo das variáveis de ambiente):

`backend/.env.example`

```
# =====
# E-CYCLE - Variaveis de ambiente do backend
# Copie este arquivo para ".env" e preencha com os seus dados.
# O arquivo .env NAO vai para o Git (esta no .gitignore).
# =====

# Porta em que a API vai rodar
PORT=3000

# String de conexao do Supabase (Postgres).
# No painel do Supabase: Project Settings > Database > Connection string > URI.
# Exemplo (troque pelos seus dados):
DATABASE_URL=postgresql://postgres.SEU_REF:SUA_SENHA@aws-0-sa-east-1.pooler.supabase.com:6543/postgres

# SSL da conexao. Deixe "true" para o Supabase.
# Use "false" apenas se for testar com um Postgres LOCAL sem SSL.
DB_SSL=true

# Numero de "rounds" do bcrypt (custo do hash da senha)
BCRYPT_ROUNDS=10

# Segredo usado para assinar os tokens JWT (troque por um valor aleatorio).
JWT_SECRET=troque-este-segredo-por-um-valor-aleatorio
# Tempo de expiracao do token
JWT_EXPIRA=8h
```

Passo 4. Agora que existe o `package.json`, instale as dependências (dentro da pasta `backend`):

```
cd backend
npm install
```

Isso cria a pasta `node_modules` (que **não** vai para o GitHub).

Parte 2 — Conexão com o banco

Este arquivo cria a conexão com o Supabase (Postgres) usando a biblioteca `pg`. Todo o resto do backend usa ele para falar com o banco.

`backend/src/config/db.js`

Conexão com o banco. O `DB_SSL` fica ligado para o Supabase.

```
// =====
// Conexao com o banco (Supabase / PostgreSQL)
// Usamos a biblioteca "pg" com um Pool de conexoes.
// O Supabase e apenas o banco: todo o CRUD e feito com SQL aqui.
// =====
const { Pool } = require('pg');
require('dotenv').config();

if (!process.env.DATABASE_URL) {
  console.warn('[db] Atencao: DATABASE_URL nao definida. Configure o arquivo .env.');
```

```
}

// O Supabase exige SSL (padrao). Para testar com um Postgres LOCAL,
// coloque DB_SSL=false no .env.
const usarSSL = process.env.DB_SSL !== 'false';

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: usarSSL ? { rejectUnauthorized: false } : false,
});

// Helper: executa uma query e devolve o resultado.
// Ex.: const r = await db.query('SELECT * FROM usuarios WHERE id_usuario = $1', [id]);
async function query(texto, params) {
  return pool.query(texto, params);
}

// Helper: pega um cliente do pool para usar transacoes (BEGIN/COMMIT).
async function getClient() {
  return pool.connect();
}

module.exports = { query, getClient, pool };
```

Parte 3 — Middlewares (segurança e apoio)

Middlewares são funções que rodam no meio do caminho de cada requisição: tratam erros, validam dados e cuidam da autenticação.

Tratamento central de erros e a classe `ErroApi`.

```
// =====  
// Tratamento centralizado de erros.  
// Qualquer erro passado por next(err) cai aqui.  
// =====  
  
// Classe simples para erros com status HTTP definido.  
class ErroApi extends Error {  
  constructor(status, mensagem) {  
    super(mensagem);  
    this.status = status;  
  }  
}  
  
// Middleware de erro (precisa ter os 4 parametros).  
function tratarErro(err, req, res, next) {  
  // Erro de violacao de UNIQUE no Postgres (ex.: email/cnpj repetido)  
  if (err.code === '23505') {  
    return res.status(409).json({ erro: 'Registro duplicado (valor unico ja existe).' });  
  }  
  // Erro de violacao de FK (ex.: id_usuario inexistente)  
  if (err.code === '23503') {  
    return res.status(400).json({ erro: 'Referencia invalida (chave estrangeira nao encontrada).' });  
  }  
  
  const status = err.status || 500;  
  const mensagem = err.status ? err.message : 'Erro interno no servidor.';  
  if (status === 500) console.error('[erro]', err);  
  res.status(status).json({ erro: mensagem });  
}  
  
// Middleware para rotas nao encontradas.  
function naoEncontrado(req, res) {  
  res.status(404).json({ erro: 'Rota nao encontrada.' });  
}  
  
module.exports = { ErroApi, tratarErro, naoEncontrado };
```

Funções que validam os dados que chegam nas rotas.

```
// =====
// Funcoes de validacao de dados de entrada.
// Retornam uma lista de erros (vazia = tudo certo).
// =====

function ehTextoValido(v) {
  return typeof v === 'string' && v.trim().length > 0;
}

function ehEmailValido(v) {
  return typeof v === 'string' && /^[^\s@]+@[^\s@]+\.[^\s@]+$/ .test(v);
}

function ehNumeroPositivo(v) {
  return !isNaN(v) && Number(v) > 0;
}

function ehInteiro(v) {
  return Number.isInteger(Number(v));
}

// Valida o corpo de criacao de usuario.
function validarUsuario(body) {
  const erros = [];
  if (!ehTextoValido(body.nome)) erros.push('nome e obrigatorio');
  if (!ehEmailValido(body.email)) erros.push('email invalido');
  if (!ehTextoValido(body.senha) || body.senha.length < 6)
    erros.push('senha deve ter ao menos 6 caracteres');
  return erros;
}

function validarEmpresa(body) {
  const erros = [];
  if (!ehTextoValido(body.nome_empresa)) erros.push('nome_empresa e obrigatorio');
  if (!ehTextoValido(body.cnpj)) erros.push('cnpj e obrigatorio');
  return erros;
}

function validarTipo(body) {
  const erros = [];
  if (!ehTextoValido(body.nome)) erros.push('nome e obrigatorio');
  return erros;
}

function validarPonto(body) {
  const erros = [];
  if (!ehTextoValido(body.nome_local)) erros.push('nome_local e obrigatorio');
  if (!ehTextoValido(body.endereco)) erros.push('endereco e obrigatorio');
  if (!ehTextoValido(body.cidade)) erros.push('cidade e obrigatorio');
  if (!ehInteiro(body.id_empresa)) erros.push('id_empresa e obrigatorio');
  return erros;
}

function validarEntrega(body) {
  const erros = [];
  if (!ehInteiro(body.id_usuario)) erros.push('id_usuario e obrigatorio');
  if (!ehInteiro(body.id_ponto)) erros.push('id_ponto e obrigatorio');
  if (!ehInteiro(body.id_tipo)) erros.push('id_tipo e obrigatorio');
  if (!ehNumeroPositivo(body.peso_residuo)) erros.push('peso_residuo deve ser maior que zero');
  return erros;
}

module.exports = {
  validarUsuario,
  validarEmpresa,
  validarTipo,
  validarPonto,
  validarEntrega,
};
```

Gera e confere o token JWT; `apenasAdmin` restringe rotas de administrador.

```
// =====
// Middlewares de SEGURANCA
// autenticar -> exige um token JWT valido (usuario logado)
// apenasAdmin -> exige que o usuario seja do tipo 'admin'
// =====
const jwt = require('jsonwebtoken');
const { ErroApi } = require('./erro');

const SEGREDO = process.env.JWT_SECRET || 'segredo-de-desenvolvimento';

// Gera um token para um usuario (usado no login e no cadastro).
function gerarToken(usuario) {
  return jwt.sign(
    { id: usuario.id_usuario, nome: usuario.nome, tipo: usuario.tipo_usuario, id_ponto: usuario.id_ponto ?? null },
    SEGREDO,
    { expiresIn: process.env.JWT_EXPIRA || '8h' }
  );
}

// Le o token do cabecalho "Authorization: Bearer <token>" e valida.
function autenticar(req, res, next) {
  const cabecalho = req.headers.authorization || '';
  const token = cabecalho.startsWith('Bearer ') ? cabecalho.slice(7) : null;
  if (!token) return next(new ErroApi(401, 'Token nao informado. Faca login.));

  try {
    // Se valido, guarda os dados do usuario em req.usuario para as rotas usarem.
    req.usuario = jwt.verify(token, SEGREDO);
    next();
  } catch {
    next(new ErroApi(401, 'Token invalido ou expirado.));
  }
}

// So deixa passar se o usuario for admin (use depois de autenticar).
function apenasAdmin(req, res, next) {
  if (req.usuario?.tipo !== 'admin') {
    return next(new ErroApi(403, 'Acesso restrito a administradores.));
  }
  next();
}

module.exports = { gerarToken, autenticar, apenasAdmin };
```

Parte 4 — Services (regra de negócio + SQL)

Os services têm o SQL e a regra de negócio. É aqui que o CRUD acontece de verdade.

Cadastro (cliente) e login (confere a senha com bcrypt).

```
// =====
// Service de AUTENTICACAO
// registrar -> cria um usuario COMUM (cliente) e ja loga
// login -> confere email + senha e devolve o token
// =====
const bcrypt = require('bcrypt');
const db = require('../config/db');
const { ErroApi } = require('../middlewares/erro');
const { gerarToken } = require('../middlewares/auth');

const ROUNDS = Number(process.env.BCRYPT_ROUNDS || 10);

// Cadastro publico: sempre cria como 'comum' (ninguem se cadastra como admin).
async function registrar({ nome, email, senha, telefone }) {
  const senhaHash = await bcrypt.hash(senha, ROUNDS);
  const r = await db.query(
    `INSERT INTO usuarios (nome, email, senha, telefone, tipo_usuario)
    VALUES ($1, $2, $3, $4, 'comum')
    RETURNING id_usuario, nome, email, tipo_usuario`,
    [nome, email, senhaHash, telefone || null]
  );
  const usuario = r.rows[0];
  return { usuario, token: gerarToken(usuario) };
}

// Login: busca o usuario pelo email, compara a senha com o hash.
async function login({ email, senha }) {
  const r = await db.query(
    `SELECT id_usuario, nome, email, senha, tipo_usuario, id_ponto FROM usuarios WHERE email = $1`,
    [email]
  );
  const usuario = r.rows[0];

  // Mensagem generica de proposito (nao revela se o email existe).
  if (!usuario) throw new ErroApi(401, 'E-mail ou senha invalidos.');
```

```
  const confere = await bcrypt.compare(senha, usuario.senha);
  if (!confere) throw new ErroApi(401, 'E-mail ou senha invalidos.');
```

```
  // Nunca devolve a senha.
  delete usuario.senha;
  return { usuario, token: gerarToken(usuario) };
}

module.exports = { registrar, login };
```

CRUD de usuários (a senha nunca é devolvida).

```
// =====
// Service de USUARIOS
// Regra de negocio + acesso ao banco (SQL puro via pg).
// A senha nunca e devolvida nas respostas.
// =====
const bcrypt = require('bcrypt');
const db = require('../config/db');
const { ErroApi } = require('../middlewares/erro');

const ROUNDS = Number(process.env.BCRYPT_ROUNDS || 10);

// Colunas que podem ser retornadas (sem a senha!)
const COLUNAS = 'id_usuario, nome, email, telefone, tipo_usuario, criado_em';

async function listar() {
  const r = await db.query(`SELECT ${COLUNAS} FROM usuarios ORDER BY id_usuario`);
  return r.rows;
}

async function buscarPorId(id) {
  const r = await db.query(
    `SELECT ${COLUNAS} FROM usuarios WHERE id_usuario = $1`,
    [id]
  );
  if (r.rows.length === 0) throw new ErroApi(404, 'Usuario nao encontrado.');
```

```
  return r.rows[0];
}

async function criar({ nome, email, senha, telefone, tipo_usuario }) {
  // Gera o HASH da senha (nunca guardamos a senha em texto).
  const senhaHash = await bcrypt.hash(senha, ROUNDS);
  const r = await db.query(
    `INSERT INTO usuarios (nome, email, senha, telefone, tipo_usuario)
    VALUES ($1, $2, $3, $4, $5)
    RETURNING ${COLUNAS}`,
    [nome, email, senhaHash, telefone || null, tipo_usuario || 'comum']
  );
  return r.rows[0];
}

async function atualizar(id, { nome, email, telefone, tipo_usuario }) {
  const r = await db.query(
    `UPDATE usuarios
    SET nome = COALESCE($2, nome),
        email = COALESCE($3, email),
        telefone = COALESCE($4, telefone),
        tipo_usuario = COALESCE($5, tipo_usuario)
    WHERE id_usuario = $1
    RETURNING ${COLUNAS}`,
    [id, nome, email, telefone, tipo_usuario]
  );
  if (r.rows.length === 0) throw new ErroApi(404, 'Usuario nao encontrado.');
```

```
  return r.rows[0];
}

async function remover(id) {
  const r = await db.query(
    `DELETE FROM usuarios WHERE id_usuario = $1 RETURNING id_usuario`,
    [id]
  );
  if (r.rows.length === 0) throw new ErroApi(404, 'Usuario nao encontrado.');
```

```
  return { id_usuario: r.rows[0].id_usuario };
}

module.exports = { listar, buscarPorId, criar, atualizar, remover };
```

CRUD de empresas.

```
// =====
// Service de EMPRESAS RECICLADORAS
// =====
const db = require('../config/db');
const { ErroApi } = require('../middlewares/erro');

async function listar() {
  const r = await db.query('SELECT * FROM empresas ORDER BY id_empresa');
  return r.rows;
}

async function buscarPorId(id) {
  const r = await db.query('SELECT * FROM empresas WHERE id_empresa = $1', [id]);
  if (r.rows.length === 0) throw new ErroApi(404, 'Empresa nao encontrada.');
  return r.rows[0];
}

async function criar({ nome_empresa, cnpj, contato }) {
  const r = await db.query(
    `INSERT INTO empresas (nome_empresa, cnpj, contato)
    VALUES ($1, $2, $3) RETURNING *`,
    [nome_empresa, cnpj, contato || null]
  );
  return r.rows[0];
}

async function atualizar(id, { nome_empresa, cnpj, contato }) {
  const r = await db.query(
    `UPDATE empresas
    SET nome_empresa = COALESCE($2, nome_empresa),
        cnpj = COALESCE($3, cnpj),
        contato = COALESCE($4, contato)
    WHERE id_empresa = $1
    RETURNING *`,
    [id, nome_empresa, cnpj, contato]
  );
  if (r.rows.length === 0) throw new ErroApi(404, 'Empresa nao encontrada.');
  return r.rows[0];
}

async function remover(id) {
  const r = await db.query(
    `DELETE FROM empresas WHERE id_empresa = $1 RETURNING id_empresa`,
    [id]
  );
  if (r.rows.length === 0) throw new ErroApi(404, 'Empresa nao encontrada.');
  return { id_empresa: r.rows[0].id_empresa };
}

module.exports = { listar, buscarPorId, criar, atualizar, remover };
```


CRUD de tipos de resíduo.

```
// =====
// Service de TIPOS DE RESIDUO
// =====
const db = require('../config/db');
const { ErroApi } = require('../middlewares/erro');

async function listar() {
  const r = await db.query('SELECT * FROM tipos_residuo ORDER BY id_tipo');
  return r.rows;
}

async function buscarPorId(id) {
  const r = await db.query('SELECT * FROM tipos_residuo WHERE id_tipo = $1', [id]);
  if (r.rows.length === 0) throw new ErroApi(404, 'Tipo de residuo nao encontrado.');
```

```
  return r.rows[0];
}

async function criar({ nome, descricao }) {
  const r = await db.query(
    'INSERT INTO tipos_residuo (nome, descricao) VALUES ($1, $2) RETURNING *',
    [nome, descricao || null]
  );
  return r.rows[0];
}

async function atualizar(id, { nome, descricao }) {
  const r = await db.query(
    `UPDATE tipos_residuo
      SET nome = COALESCE($2, nome),
          descricao = COALESCE($3, descricao)
      WHERE id_tipo = $1 RETURNING *`,
    [id, nome, descricao]
  );
  if (r.rows.length === 0) throw new ErroApi(404, 'Tipo de residuo nao encontrado.');
```

```
  return r.rows[0];
}

async function remover(id) {
  const r = await db.query(
    'DELETE FROM tipos_residuo WHERE id_tipo = $1 RETURNING id_tipo',
    [id]
  );
  if (r.rows.length === 0) throw new ErroApi(404, 'Tipo de residuo nao encontrado.');
```

```
  return { id_tipo: r.rows[0].id_tipo };
}

module.exports = { listar, buscarPorId, criar, atualizar, remover };
```

backend/src/services/pontos.service.js

CRUD de pontos de coleta (com JOIN da empresa).

```
// =====
// Service de PONTOS DE COLETA
// Cada ponto pertence a uma empresa (FK id_empresa).
// A listagem traz o nome da empresa (JOIN).
// =====
const db = require('../config/db');
const { ErroApi } = require('../middlewares/erro');

async function listar() {
  const r = await db.query(
    `SELECT pc.id_ponto, pc.nome_local, pc.endereco, pc.cidade,
      pc.id_empresa, e.nome_empresa
      FROM pontos_coleta pc
      JOIN empresas e ON e.id_empresa = pc.id_empresa
      ORDER BY pc.id_ponto`
  );
  return r.rows;
}

async function buscarPorId(id) {
  const r = await db.query(
    `SELECT pc.*, e.nome_empresa
      FROM pontos_coleta pc
      JOIN empresas e ON e.id_empresa = pc.id_empresa
      WHERE pc.id_ponto = $1`,
    [id]
  );
  if (r.rows.length === 0) throw new ErroApi(404, 'Ponto de coleta nao encontrado.');
```

```
  return r.rows[0];
}

async function criar({ nome_local, endereco, cidade, id_empresa }) {
  const r = await db.query(
    `INSERT INTO pontos_coleta (nome_local, endereco, cidade, id_empresa)
      VALUES ($1, $2, $3, $4) RETURNING *`,
    [nome_local, endereco, cidade, id_empresa]
  );
  return r.rows[0];
}

async function atualizar(id, { nome_local, endereco, cidade, id_empresa }) {
  const r = await db.query(
    `UPDATE pontos_coleta
      SET nome_local = COALESCE($2, nome_local),
          endereco = COALESCE($3, endereco),
          cidade = COALESCE($4, cidade),
          id_empresa = COALESCE($5, id_empresa)
      WHERE id_ponto = $1 RETURNING *`,
    [id, nome_local, endereco, cidade, id_empresa]
  );
  if (r.rows.length === 0) throw new ErroApi(404, 'Ponto de coleta nao encontrado.');
```

```
  return r.rows[0];
}

async function remover(id) {
  const r = await db.query(
    `DELETE FROM pontos_coleta WHERE id_ponto = $1 RETURNING id_ponto`,
    [id]
  );
  if (r.rows.length === 0) throw new ErroApi(404, 'Ponto de coleta nao encontrado.');
```

```
  return { id_ponto: r.rows[0].id_ponto };
}

module.exports = { listar, buscarPorId, criar, atualizar, remover };
```

backend/src/services/entregas.service.js

O descarte: solicitar, confirmar (gera pontos) e recusar.

```
// =====
// Service de ENTREGAS (o "descarte") + GAMIFICACAO
//
// Fluxo:
// 1) cliente SOLICITA -> status 'solicitado' (NAO pontua)
// 2) admin do ponto CONFIRMA -> status 'confirmado' + gera pontos
//    (ou RECUSA -> status 'recusado')
//
// Regras de gamificacao (so aplicadas na confirmacao):
// pontos = peso * 10 | nivel = floor(pontos/100) + 1
// =====
const db = require('../config/db');
const { ErroApi } = require('../middlewares/erro');
```

```
const PONTOS_POR_KG = 10;
const PONTOS_POR_NIVEL = 100;
const calcularPontos = (peso) => Math.round(Number(peso) * PONTOS_POR_KG);
const calcularNivel = (total) => Math.floor(total / PONTOS_POR_NIVEL) + 1;
```

```

// SELECT base com os nomes (JOINS), usado nas listagens.
const SELECT_BASE = `
  SELECT e.id_entrega, e.peso_residuo, e.status, e.data_entrega, e.data_confirmacao,
         u.id_usuario, u.nome AS nome_usuario,
         pc.id_ponto, pc.nome_local,
         tr.id_tipo, tr.nome AS nome_tipo
  FROM entregas e
  JOIN usuarios u      ON u.id_usuario = e.id_usuario
  JOIN pontos_coleta pc ON pc.id_ponto  = e.id_ponto
  JOIN tipos_residuo tr ON tr.id_tipo   = e.id_tipo`;

// Cliente solicita o descarte (sem pontuar).
async function solicitar({ id_usuario, id_ponto, id_tipo, peso_residuo }) {
  const r = await db.query(
    `INSERT INTO entregas (id_usuario, id_ponto, id_tipo, peso_residuo, status)
     VALUES ($1, $2, $3, $4, 'solicitado')
     RETURNING id_entrega, peso_residuo, status, data_entrega`,
    [id_usuario, id_ponto, id_tipo, peso_residuo]
  );
  return r.rows[0];
}

// Lista solicitacoes PENDENTES. Se filtroPonto for informado
// (admin de ponto), so as do ponto dele.
async function listarPendentes(filtroPonto) {
  const where = filtroPonto
    ? `WHERE e.status = 'solicitado' AND e.id_ponto = $1`
    : `WHERE e.status = 'solicitado'`;
  const params = filtroPonto ? [filtroPonto] : [];
  const r = await db.query(`${SELECT_BASE} ${where} ORDER BY e.data_entrega ASC`, params);
  return r.rows;
}

// Historico geral (todos os status). filtroPonto opcional.
async function listar(filtroPonto) {
  const where = filtroPonto ? `WHERE e.id_ponto = $1` : '';
  const params = filtroPonto ? [filtroPonto] : [];
  const r = await db.query(`${SELECT_BASE} ${where} ORDER BY e.data_entrega DESC`, params);
  return r.rows;
}

// Busca uma entrega (para checagens).
async function buscar(id) {
  const r = await db.query(`SELECT * FROM entregas WHERE id_entrega = $1`, [id]);
  if (r.rows.length === 0) throw new ErroApi(404, 'Descarte nao encontrado.');
```

```

  return r.rows[0];
}

// Admin do ponto CONFIRMA -> gera a pontuacao (transacao).
// adminIdPonto: se o admin for de ponto, so pode confirmar do proprio ponto.
async function confirmar(idEntrega, adminIdPonto) {
  const entrega = await buscar(idEntrega);
  if (entrega.status !== 'solicitado')
    throw new ErroApi(409, `Este descarte ja foi ${entrega.status}.`);
  if (adminIdPonto && entrega.id_ponto !== adminIdPonto)
    throw new ErroApi(403, 'Este descarte e de outro ponto de coleta.');
```

```

  const client = await db.getClient();
  try {
    await client.query('BEGIN');
    await client.query(
      `UPDATE entregas SET status = 'confirmado', data_confirmacao = NOW() WHERE id_entrega = $1`,
      [idEntrega]
    );
    const pontosGanhos = calcularPontos(entrega.peso_residuo);
    const upsert = await client.query(
      `INSERT INTO pontuacao (id_usuario, pontos, nivel)
       VALUES ($1, $2, 1)
       ON CONFLICT (id_usuario)
       DO UPDATE SET pontos = pontuacao.pontos + EXCLUDED.pontos
       RETURNING pontos`,
      [entrega.id_usuario, pontosGanhos]
    );
    const pontosTotais = upsert.rows[0].pontos;
    const nivel = calcularNivel(pontosTotais);
    await client.query(
      `UPDATE pontuacao SET nivel = $2, data_registro = NOW() WHERE id_usuario = $1`,
      [entrega.id_usuario, nivel]
    );
    await client.query('COMMIT');
    return { id_entrega: idEntrega, status: 'confirmado',
      gamificacao: { pontos_ganhos: pontosGanhos, pontos_totais: pontosTotais, nivel } };
  } catch (err) {
    await client.query('ROLLBACK');
    throw err;
  } finally {
    client.release();
  }
}

// Admin do ponto RECUSA (sem pontuar).
async function recusar(idEntrega, adminIdPonto) {

```

```

const entrega = await buscar(idEntrega);
if (entrega.status !== 'solicitado')
  throw new ErroApi(409, `Este descarte ja foi ${entrega.status}.`);
if (adminIdPonto && entrega.id_ponto !== adminIdPonto)
  throw new ErroApi(403, 'Este descarte e de outro ponto de coleta.');
```

```

await db.query(`UPDATE entregas SET status = 'recusado' WHERE id_entrega = $1`, [idEntrega]);
return { id_entrega: idEntrega, status: 'recusado' };
}

module.exports = { solicitar, listarPendentes, listar, confirmar, recusar };

```

backend/src/services/perfil.service.js

Dados e histórico do próprio usuário logado.

```

// =====
// Service de PERFIL – dados do PRÓPRIO usuário logado.
// Sempre usa o id vindo do token (nunca do corpo da requisicao).
// =====
const db = require('../config/db');

// Info do usuário + pontuacao acumulada.
async function meusDados(idUsuario) {
  const usuario = await db.query(
    `SELECT id_usuario, nome, email, telefone, tipo_usuario FROM usuarios WHERE id_usuario = $1`,
    [idUsuario]
  );
  const pont = await db.query(
    `SELECT pontos, nivel FROM pontuacao WHERE id_usuario = $1`,
    [idUsuario]
  );
  return {
    usuario: usuario.rows[0],
    pontuacao: pont.rows[0] || { pontos: 0, nivel: 1 },
  };
}

// Historico de entregas do proprio usuario.
async function minhasEntregas(idUsuario) {
  const r = await db.query(
    `SELECT e.id_entrega, e.peso_residuo, e.status, e.data_entrega,
      pc.nome_local, tr.nome AS nome_tipo
      FROM entregas e
      JOIN pontos_coleta pc ON pc.id_ponto = e.id_ponto
      JOIN tipos_residuo tr ON tr.id_tipo = e.id_tipo
      WHERE e.id_usuario = $1
      ORDER BY e.data_entrega DESC`,
    [idUsuario]
  );
  return r.rows;
}

module.exports = { meusDados, minhasEntregas };

```

As 4 consultas de relatório (só entregas confirmadas).

```
// =====
// Service de RELATORIOS (ETAPA 6)
// As mesmas consultas de banco/03_consultas_relatorios.sql.
// =====
const db = require('../config/db');

// 1) Quantidade de residuos coletados (total geral)
async function totaisColetados() {
  const r = await db.query(
    `SELECT COUNT(*) AS total_entregas,
      COALESCE(SUM(peso_residuo), 0) AS peso_total_kg
    FROM entregas`
  );
  return r.rows[0];
}

// 2) Usuarios que mais reciclam (ranking)
async function usuariosQueMaisReciclam() {
  const r = await db.query(
    `SELECT u.id_usuario, u.nome,
      COALESCE(p.pontos, 0) AS pontos,
      COALESCE(p.nivel, 1) AS nivel,
      COUNT(e.id_entrega) AS total_entregas,
      COALESCE(SUM(e.peso_residuo), 0) AS peso_total_kg
    FROM usuarios u
    LEFT JOIN pontuacao p ON p.id_usuario = u.id_usuario
    LEFT JOIN entregas e ON e.id_usuario = u.id_usuario
    GROUP BY u.id_usuario, u.nome, p.pontos, p.nivel
    ORDER BY pontos DESC, peso_total_kg DESC`
  );
  return r.rows;
}

// 3) Pontos de coleta mais utilizados
async function pontosMaisUtilizados() {
  const r = await db.query(
    `SELECT pc.id_ponto, pc.nome_local, pc.cidade,
      COUNT(e.id_entrega) AS total_entregas,
      COALESCE(SUM(e.peso_residuo), 0) AS peso_total_kg
    FROM pontos_coleta pc
    LEFT JOIN entregas e ON e.id_ponto = pc.id_ponto
    GROUP BY pc.id_ponto, pc.nome_local, pc.cidade
    ORDER BY total_entregas DESC`
  );
  return r.rows;
}

// 4) Tipos de residuos mais descartados
async function tiposMaisDescartados() {
  const r = await db.query(
    `SELECT tr.id_tipo, tr.nome,
      COUNT(e.id_entrega) AS total_entregas,
      COALESCE(SUM(e.peso_residuo), 0) AS peso_total_kg
    FROM tipos_residuo tr
    LEFT JOIN entregas e ON e.id_tipo = tr.id_tipo
    GROUP BY tr.id_tipo, tr.nome
    ORDER BY total_entregas DESC`
  );
  return r.rows;
}

module.exports = {
  totaisColetados,
  usuariosQueMaisReciclam,
  pontosMaisUtilizados,
  tiposMaisDescartados,
};
```

Parte 5 — Controllers (recebem as requisições)

Os controllers recebem a requisição (`req`), chamam o service e devolvem a resposta (`res`).

backend/src/controllers/auth.controller.js

Cadastro e login.

```
// =====  
// Controller de AUTENTICACAO  
// =====  
const service = require('../services/auth.service');  
const { validarUsuario } = require('../middlewares/validacao');  
  
async function registrar(req, res, next) {  
  try {  
    const erros = validarUsuario(req.body);  
    if (erros.length) return res.status(400).json({ erros });  
    const resultado = await service.registrar(req.body);  
    res.status(201).json(resultado);  
  } catch (e) { next(e); }  
}  
  
async function login(req, res, next) {  
  try {  
    const { email, senha } = req.body;  
    if (!email || !senha) return res.status(400).json({ erro: 'Informe email e senha.' });  
    const resultado = await service.login({ email, senha });  
    res.json(resultado);  
  } catch (e) { next(e); }  
}  
  
module.exports = { registrar, login };
```

backend/src/controllers/usuarios.controller.js

Usuários.

```
// =====  
// Controller de USUARIOS  
// =====  
const service = require('../services/usuarios.service');  
const { validarUsuario } = require('../middlewares/validacao');  
  
async function listar(req, res, next) {  
  try {  
    res.json(await service.listar());  
  } catch (e) { next(e); }  
}  
  
async function buscar(req, res, next) {  
  try {  
    res.json(await service.buscarPorId(req.params.id));  
  } catch (e) { next(e); }  
}  
  
async function criar(req, res, next) {  
  try {  
    const erros = validarUsuario(req.body);  
    if (erros.length) return res.status(400).json({ erros });  
    const novo = await service.criar(req.body);  
    res.status(201).json(novo);  
  } catch (e) { next(e); }  
}  
  
async function atualizar(req, res, next) {  
  try {  
    res.json(await service.atualizar(req.params.id, req.body));  
  } catch (e) { next(e); }  
}  
  
async function remover(req, res, next) {  
  try {  
    res.json(await service.remover(req.params.id));  
  } catch (e) { next(e); }  
}  
  
module.exports = { listar, buscar, criar, atualizar, remover };
```

backend/src/controllers/empresas.controller.js

Empresas.

```
// =====  
// Controller de EMPRESAS  
// =====  
const service = require('../services/empresas.service');  
const { validarEmpresa } = require('../middlewares/validacao');  
  
async function listar(req, res, next) {  
  try { res.json(await service.listar()); } catch (e) { next(e); }  
}  
async function buscar(req, res, next) {  
  try { res.json(await service.buscarPorId(req.params.id)); } catch (e) { next(e); }  
}  
async function criar(req, res, next) {  
  try {  
    const erros = validarEmpresa(req.body);  
    if (erros.length) return res.status(400).json({ erros });  
    res.status(201).json(await service.criar(req.body));  
  } catch (e) { next(e); }  
}  
async function atualizar(req, res, next) {  
  try { res.json(await service.atualizar(req.params.id, req.body)); } catch (e) { next(e); }  
}  
async function remover(req, res, next) {  
  try { res.json(await service.remover(req.params.id)); } catch (e) { next(e); }  
}  
  
module.exports = { listar, buscar, criar, atualizar, remover };
```

backend/src/controllers/tipos.controller.js

Tipos de resíduo.

```
// =====  
// Controller de TIPOS DE RESIDUO  
// =====  
const service = require('../services/tipos.service');  
const { validarTipo } = require('../middlewares/validacao');  
  
async function listar(req, res, next) {  
  try { res.json(await service.listar()); } catch (e) { next(e); }  
}  
async function buscar(req, res, next) {  
  try { res.json(await service.buscarPorId(req.params.id)); } catch (e) { next(e); }  
}  
async function criar(req, res, next) {  
  try {  
    const erros = validarTipo(req.body);  
    if (erros.length) return res.status(400).json({ erros });  
    res.status(201).json(await service.criar(req.body));  
  } catch (e) { next(e); }  
}  
async function atualizar(req, res, next) {  
  try { res.json(await service.atualizar(req.params.id, req.body)); } catch (e) { next(e); }  
}  
async function remover(req, res, next) {  
  try { res.json(await service.remover(req.params.id)); } catch (e) { next(e); }  
}  
  
module.exports = { listar, buscar, criar, atualizar, remover };
```

backend/src/controllers/pontos.controller.js

Pontos de coleta.

```
// =====  
// Controller de PONTOS DE COLETA  
// =====  
const service = require('../services/pontos.service');  
const { validarPonto } = require('../middlewares/validacao');  
  
async function listar(req, res, next) {  
  try { res.json(await service.listar()); } catch (e) { next(e); }  
}  
async function buscar(req, res, next) {  
  try { res.json(await service.buscarPorId(req.params.id)); } catch (e) { next(e); }  
}  
async function criar(req, res, next) {  
  try {  
    const erros = validarPonto(req.body);  
    if (erros.length) return res.status(400).json({ erros });  
    res.status(201).json(await service.criar(req.body));  
  } catch (e) { next(e); }  
}  
async function atualizar(req, res, next) {  
  try { res.json(await service.atualizar(req.params.id, req.body)); } catch (e) { next(e); }  
}  
async function remover(req, res, next) {  
  try { res.json(await service.remover(req.params.id)); } catch (e) { next(e); }  
}  
  
module.exports = { listar, buscar, criar, atualizar, remover };
```

backend/src/controllers/entregas.controller.js

Descarte (o id do usuário vem do token).

```
// =====  
// Controller de ENTREGAS  
// =====  
const service = require('../services/entregas.service');  
const { validarEntrega } = require('../middlewares/validacao');  
  
// Cliente solicita o proprio descarte (id_usuario vem do TOKEN).  
async function solicitar(req, res, next) {  
  try {  
    const dados = {  
      id_usuario: req.usuario.id,  
      id_ponto: req.body.id_ponto,  
      id_tipo: req.body.id_tipo,  
      peso_residuo: req.body.peso_residuo,  
    };  
    const erros = validarEntrega(dados);  
    if (erros.length) return res.status(400).json({ erros });  
    res.status(201).json(await service.solicitar(dados));  
  } catch (e) { next(e); }  
}  
  
// Admin do ponto ve so o proprio ponto; admin global ve tudo.  
function pontoDoAdmin(req) {  
  return req.usuario.id_ponto || null; // null = admin global  
}  
  
async function pendentes(req, res, next) {  
  try { res.json(await service.listarPendentes(pontoDoAdmin(req))); } catch (e) { next(e); }  
}  
  
async function listar(req, res, next) {  
  try { res.json(await service.listar(pontoDoAdmin(req))); } catch (e) { next(e); }  
}  
  
async function confirmar(req, res, next) {  
  try { res.json(await service.confirmar(req.params.id, pontoDoAdmin(req))); } catch (e) { next(e); }  
}  
  
async function recusar(req, res, next) {  
  try { res.json(await service.recusar(req.params.id, pontoDoAdmin(req))); } catch (e) { next(e); }  
}  
  
module.exports = { solicitar, pendentes, listar, confirmar, recusar };
```


backend/src/controllers/perfil.controller.js

Perfil do usuário logado.

```
// =====  
// Controller de PERFIL  
// =====  
const service = require('../services/perfil.service');  
  
async function meusDados(req, res, next) {  
  try {  
    // req.usuario.id vem do token (middleware autenticar)  
    res.json(await service.meusDados(req.usuario.id));  
  } catch (e) { next(e); }  
}  
  
async function minhasEntregas(req, res, next) {  
  try {  
    res.json(await service.minhasEntregas(req.usuario.id));  
  } catch (e) { next(e); }  
}  
  
module.exports = { meusDados, minhasEntregas };
```

backend/src/controllers/relatorios.controller.js

Relatórios.

```
// =====  
// Controller de RELATORIOS  
// =====  
const service = require('../services/relatorios.service');  
  
async function totais(req, res, next) {  
  try { res.json(await service.totaisColetados()); } catch (e) { next(e); }  
}  
async function ranking(req, res, next) {  
  try { res.json(await service.usuariosQueMaisReciclam()); } catch (e) { next(e); }  
}  
async function pontos(req, res, next) {  
  try { res.json(await service.pontosMaisUtilizados()); } catch (e) { next(e); }  
}  
async function tipos(req, res, next) {  
  try { res.json(await service.tiposMaisDescartados()); } catch (e) { next(e); }  
}  
  
module.exports = { totais, ranking, pontos, tipos };
```

Parte 6 — Rotas (os endereços da API)

As rotas definem as URLs da API e ligam cada uma ao controller, aplicando a proteção por perfil.

backend/src/routes/auth.routes.js

Rotas públicas: cadastro e login.

```
// Rotas publicas de autenticacao (nao exigem token).  
const express = require('express');  
const router = express.Router();  
const ctrl = require('../controllers/auth.controller');  
  
router.post('/register', ctrl.registrar); // cadastro de cliente  
router.post('/login', ctrl.login); // login  
  
module.exports = router;
```

backend/src/routes/usuarios.routes.js

Usuários (somente admin).

```
const express = require('express');  
const router = express.Router();  
const ctrl = require('../controllers/usuarios.controller');  
const { apenasAdmin } = require('../middlewares/auth');  
  
// Gestao de usuarios: somente admin.  
router.get('/', apenasAdmin, ctrl.listar);  
router.get('/:id', apenasAdmin, ctrl.buscar);  
router.post('/', apenasAdmin, ctrl.criar);  
router.put('/:id', apenasAdmin, ctrl.atualizar);  
router.delete('/:id', apenasAdmin, ctrl.remover);  
  
module.exports = router;
```

backend/src/routes/empresas.routes.js

Empresas (somente admin).

```
const express = require('express');
const router = express.Router();
const ctrl = require('../controllers/empresas.controller');
const { apenasAdmin } = require('../middlewares/auth');

// Empresas: gestao so admin.
router.get('/', apenasAdmin, ctrl.listar);
router.get('/:id', apenasAdmin, ctrl.buscar);
router.post('/', apenasAdmin, ctrl.criar);
router.put('/:id', apenasAdmin, ctrl.atualizar);
router.delete('/:id', apenasAdmin, ctrl.remover);

module.exports = router;
```

backend/src/routes/tipos.routes.js

Tipos (listar: logado; gestão: admin).

```
const express = require('express');
const router = express.Router();
const ctrl = require('../controllers/tipos.controller');
const { apenasAdmin } = require('../middlewares/auth');

// Listar/buscar: qualquer logado (o cliente precisa para escolher no descarte).
router.get('/', ctrl.listar);
router.get('/:id', ctrl.buscar);
// Gestao: so admin.
router.post('/', apenasAdmin, ctrl.criar);
router.put('/:id', apenasAdmin, ctrl.atualizar);
router.delete('/:id', apenasAdmin, ctrl.remover);

module.exports = router;
```

backend/src/routes/pontos.routes.js

Pontos (listar: logado; gestão: admin).

```
const express = require('express');
const router = express.Router();
const ctrl = require('../controllers/pontos.controller');
const { apenasAdmin } = require('../middlewares/auth');

// Listar/buscar: qualquer logado (o cliente precisa para escolher no descarte).
router.get('/', ctrl.listar);
router.get('/:id', ctrl.buscar);
// Gestao: so admin.
router.post('/', apenasAdmin, ctrl.criar);
router.put('/:id', apenasAdmin, ctrl.atualizar);
router.delete('/:id', apenasAdmin, ctrl.remover);

module.exports = router;
```

backend/src/routes/entregas.routes.js

Descarte: solicitar (logado), confirmar/recusar (admin).

```
const express = require('express');
const router = express.Router();
const ctrl = require('../controllers/entregas.controller');
const { apenasAdmin } = require('../middlewares/auth');

// Cliente (logado) solicita o proprio descarte.
router.post('/', ctrl.solicitar);

// Admin: fila de pendentes, historico, confirmar e recusar.
router.get('/pendentes', apenasAdmin, ctrl.pendentes);
router.get('/', apenasAdmin, ctrl.listar);
router.put('/:id/confirmar', apenasAdmin, ctrl.confirmar);
router.put('/:id/recusar', apenasAdmin, ctrl.recusar);

module.exports = router;
```

backend/src/routes/perfil.routes.js

Dados do próprio usuário.

```
// Rotas do proprio usuario logado (qualquer perfil autenticado).
const express = require('express');
const router = express.Router();
const ctrl = require('../controllers/perfil.controller');

router.get('/', ctrl.meusDados); // meus dados + pontuacao
router.get('/entregas', ctrl.minhasEntregas); // meu historico

module.exports = router;
```

backend/src/routes/relatorios.routes.js

Relatórios (somente admin).

```
const express = require('express');
const router = express.Router();
const ctrl = require('../controllers/relatorios.controller');
const { apenasAdmin } = require('../middlewares/auth');

// Relatorios: somente admin.
router.get('/totais', apenasAdmin, ctrl.totais);
router.get('/ranking', apenasAdmin, ctrl.ranking);
router.get('/pontos', apenasAdmin, ctrl.pontos);
router.get('/tipos', apenasAdmin, ctrl.tipos);

module.exports = router;
```

Parte 7 — O servidor (junta tudo)

O `server.js` é o coração: liga os middlewares, monta as rotas e sobe a API.

Ponto de entrada da aplicação.

```
// =====
// E-CYCLE - Servidor da API (Node.js + Express)
// Ponto de entrada: monta middlewares e rotas.
// =====
require('dotenv').config();
const express = require('express');
const cors = require('cors');

const { tratarErro, naoEncontrado } = require('./middlewares/erro');
const { autenticar } = require('./middlewares/auth');

// Rotas
const authRoutes = require('./routes/auth.routes');
const perfilRoutes = require('./routes/perfil.routes');
const usuariosRoutes = require('./routes/usuarios.routes');
const empresasRoutes = require('./routes/empresas.routes');
const tiposRoutes = require('./routes/tipos.routes');
const pontosRoutes = require('./routes/pontos.routes');
const entregasRoutes = require('./routes/entregas.routes');
const relatoriosRoutes = require('./routes/relatorios.routes');

const app = express();
const PORT = process.env.PORT || 3000;

// --- Middlewares globais ---
app.use(cors()); // libera o acesso do frontend
app.use(express.json()); // interpreta o corpo JSON das requisicoes

// --- Rota de saude (publica) ---
app.get('/', (req, res) => {
  res.json({ app: 'E-Cycle API', status: 'ok', versao: '2.0.0' });
});

// --- Rotas PUBLICAS (cadastro e login) ---
app.use('/api/auth', authRoutes);

// --- A partir daqui, tudo exige token (usuario logado) ---
app.use('/api', autenticar);

// --- Rotas protegidas ---
app.use('/api/perfil', perfilRoutes); // qualquer logado (dados proprios)
app.use('/api/usuarios', usuariosRoutes); // admin
app.use('/api/empresas', empresasRoutes); // admin
app.use('/api/tipos', tiposRoutes); // listar: logado | gestao: admin
app.use('/api/pontos', pontosRoutes); // listar: logado | gestao: admin
app.use('/api/entregas', entregasRoutes); // registrar: logado | historico: admin
app.use('/api/relatorios', relatoriosRoutes); // admin

// --- Rotas inexistentes e tratamento de erros ---
app.use(naoEncontrado);
app.use(tratarErro);

app.listen(PORT, () => {
  console.log(`E-Cycle API rodando em http://localhost:${PORT}`);
});
```

Parte 8 — Configurar o .env e rodar a API

Passo 1. Na pasta `backend`, crie o arquivo `.env` a partir do modelo:

```
copy .env.example .env      # Windows
# cp .env.example .env      # Linux / Mac
```

Passo 2. Abra o `.env` e preencha (cole a sua string do Supabase na `DATABASE_URL` e troque a senha):

```
PORT=3000
DATABASE_URL=postgresql://postgres.SEU_REF:SUA_SENHA@aws-0-...pooler.supabase.com:5432/postgres
DB_SSL=true
JWT_SECRET=um-valor-secreto-qualquer
JWT_EXPIRA=8h
BCRYPT_ROUNDS=10
```

O .env é secreto

Ele tem a senha do banco e **não** vai para o GitHub (o `.gitignore` já cuida disso). Nunca compartilhe.

Passo 3. Suba a API:

```
npm start
```

Deve aparecer: `E-Cycle API rodando em http://localhost:3000`.

Passo 4. Teste no navegador: acesse `http://localhost:3000/` — deve responder um JSON com `"status": "ok"`. Depois teste o login (em outro terminal):

```
curl -X POST http://localhost:3000/api/auth/login -H "Content-Type: application/json" -d '{"email":"diego@email.com","senha":"admin123"}'
```

Como saber que deu certo

Se o login devolver um **token** (um texto longo), a API está funcionando e conectada ao banco. Backend concluído!

Entrega e avaliação

Entregue: o backend commitado no seu repositório (pasta `backend/`) e um print da API respondendo o login com token.

- Estrutura de pastas correta (config, middlewares, services, controllers, routes).
- API sobe sem erros e conecta ao Supabase.
- Login retorna token; rotas de admin protegidas.
- `.env` e `node_modules` fora do Git.